Ex No: 04 PACKET SNIFFING USING RAW SOCKETS

AIM:

To write a Python program using raw sockets to capture and display network packets and their details, such as source IP address, destination IP address, protocol, and TTL.

ALGORITHM:

1. Start the program.
2. Create a raw socket using Python's socket module.
3. Bind or listen on the network interface as supported by the operating system.
4. Receive packets from the network.
5. Extract the IP header from each packet.
6. Identify the protocol, such as TCP, UDP or ICMP.
7. Extract the source and destination IP addresses.
8. Display the packet information.
9. Repeat until the user terminates the program.
10. Close the socket and stop the program.

PROGRAM:

```
import socket
import struct
def packet_sniffer():
    host = socket.gethostbyname(socket.gethostname())
    try:
        sniffer = socket.socket(socket.AF_INET,
                                socket.SOCK_RAW,
                                socket.IPPROTO_IP)

        sniffer.bind((host, 0))
        sniffer.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
        sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)
        print("Packet Sniffer Started...")
```

```
        print ("Local IP Address: ", host)
        print ("Press Ctrl+C to stop.\n")
        while True:
            packet, address = sniffer.recvfrom(65535)
            if len(packet) < 20:
                continue
            ip_header = packet[:20]
            fields = struct.unpack("!BBHHHBBH4s4s", ip_header)
            version_ihl = fields[0]
            ttl = fields[5]
            protocol = fields[6]
            src_ip = socket.inet_ntoa(fields[8])
            dest_ip = socket.inet_ntoa(fields[9])
            header_length = (version_ihl & 0x0F) * 4
            protocols = { 1: "ICMP",
                          6: "TCP",
                          17: "UDP"
            }
            print ("-" * 40)
            print ("Source IP   :", src_ip)
            print ("Destination IP:", dest_ip)
            print ("Protocol    :", protocols.get(protocol, "Other"))
            print ("Packet Size :", len(packet), "bytes")
            print ("IP Header Size :", header_length, "bytes")

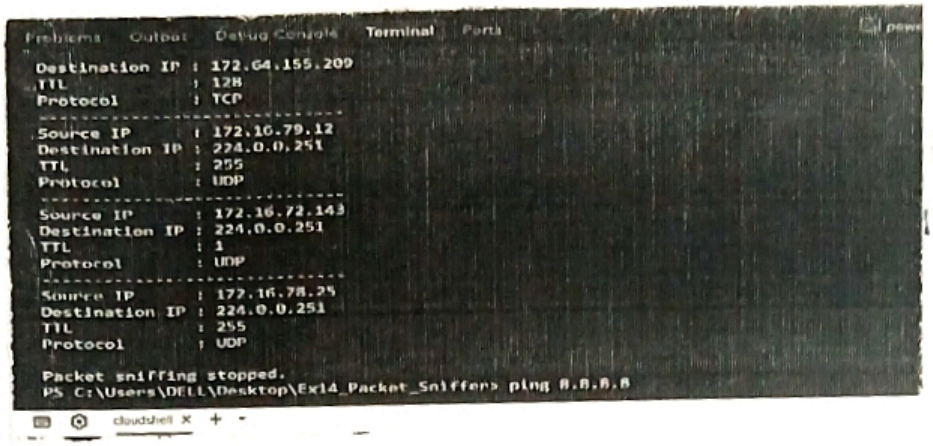
    except PermissionError:
        print ("Permission denied. Run VS Code as Administrator.")
    except OSError as e:
        print ("Socket error:", e)
    except KeyboardInterrupt:
        print ("\nPacket sniffing stopped.")
    finally:
        try:
            sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
```

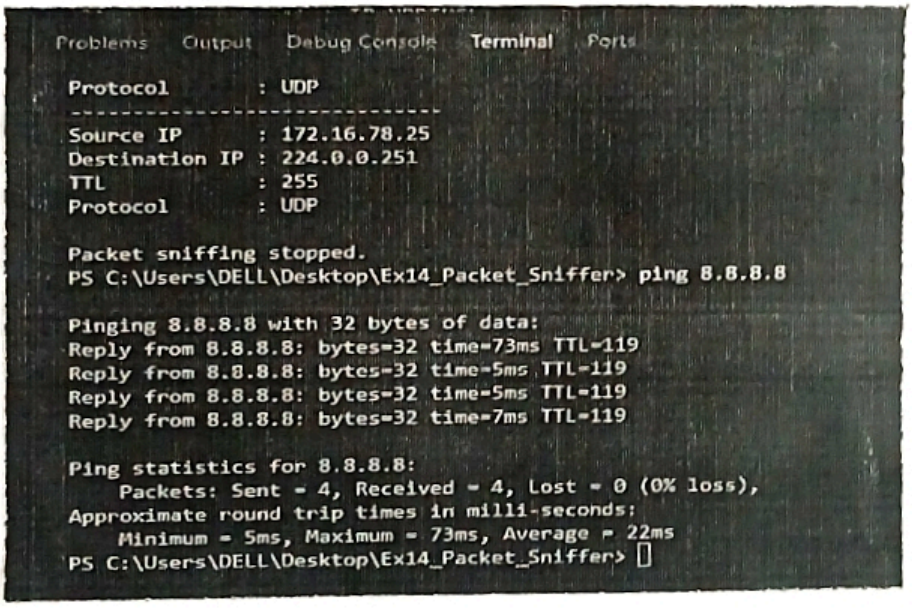
```
        sniffer.close()
    except (NameError, OSError):
        pass

if __name__ == "__main__":
    packet_sniffer()
```

RESULT:

Thus, a packet-sniffing program was implemented using Python raw sockets. The program captures IPv4 packets and displays important information, including source IP address, destination IP address, protocol type, TTL and packet size.


Problems Output Debug Console Terminal Ports
Destination IP : 172.64.155.209
TTL : 128
Protocol : TCP
Source IP : 172.16.79.12
Destination IP : 224.0.0.251
TTL : 255
Protocol : UDP
Source IP : 172.16.72.143
Destination IP : 224.0.0.251
TTL : 1
Protocol : UDP
Source IP : 172.16.78.25
Destination IP : 224.0.0.251
TTL : 255
Protocol : UDP
Packet sniffing stopped.
PS C:\Users\DELL\Desktop\Ex14_Packet_Sniffer> ping 8.8.8.8
cloudshell


Problems Output Debug Console Terminal Ports
Protocol : UDP
Source IP : 172.16.78.25
Destination IP : 224.0.0.251
TTL : 255
Protocol : UDP
Packet sniffing stopped.
PS C:\Users\DELL\Desktop\Ex14_Packet_Sniffer> ping 8.8.8.8
Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=73ms TTL=119
Reply from 8.8.8.8: bytes=32 time=5ms TTL=119
Reply from 8.8.8.8: bytes=32 time=5ms TTL=119
Reply from 8.8.8.8: bytes=32 time=7ms TTL=119
Ping statistics for 8.8.8.8:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 5ms, Maximum = 73ms, Average = 22ms
PS C:\Users\DELL\Desktop\Ex14_Packet_Sniffer>